



Universidad de
los Andes



**FACULTAD
DE INGENIERÍA
Y CIENCIAS
APLICADAS**

ARQUITECTURA DE COMPUTADORES

Proyecto 1

Grupo 15:

Integrantes:

Martin Heiremans

Felix Svensson

Juan Pablo Saavedra

Profesor:

Jorge Gómez Mir

Santiago, 8 de septiembre de 2026

ÍNDICE

1. INTRODUCCIÓN

2. DISEÑO GENERAL Y ARQUITECTURA

Figura 1: Diagrama de bloques de la calculadora

3. TABLAS DE VERDAD, K-MAPS Y EXPRESIONES BOOLEANAS

3.1 Multiplexor 2:1

3.2 Sumador completo

3.3 Sumador y restador de 4 bits

3.4 Decodificador de control

4. IMPLEMENTACIÓN MEDIANTE COMPUERTAS

5. SIMULACIÓN Y RESULTADOS

5.1 Estrategia de verificación

5.2 Resultado de simulación

5.3 Síntesis

5.4 Pruebas en placa

1. INTRODUCCIÓN

Este informe documenta el diseño y la implementación de una calculadora de 4 bits descrita en Verilog y ejecutada sobre una FPGA Lattice iCE40 HX1K, montada en la placa Nandland Go Board.

El propósito del proyecto es recorrer el flujo completo de diseño digital: partir de la especificación de un circuito mediante tablas de verdad y mapas de Karnaugh, traducir esa especificación a una descripción en Verilog, verificar su comportamiento por simulación, y finalmente sintetizarla e implementarla en un dispositivo físico. Cada una de esas etapas se documenta en las secciones siguientes.

La restricción central del trabajo es que toda la lógica combinacional debe construirse exclusivamente con compuertas lógicas. No se permite emplear los operadores aritméticos, relacionales ni de desplazamiento del lenguaje, ni las construcciones condicionales de alto nivel.

El documento se organiza de la siguiente manera: la sección 1 presenta el diseño general y la arquitectura de la calculadora, junto con las decisiones que permitieron compartir hardware entre operaciones. La sección 2 desarrolla las tablas de verdad, los mapas de Karnaugh y las expresiones booleanas de cada bloque. La sección 3 muestra la implementación de esas expresiones mediante compuertas, y la sección 4 documenta los resultados obtenidos.

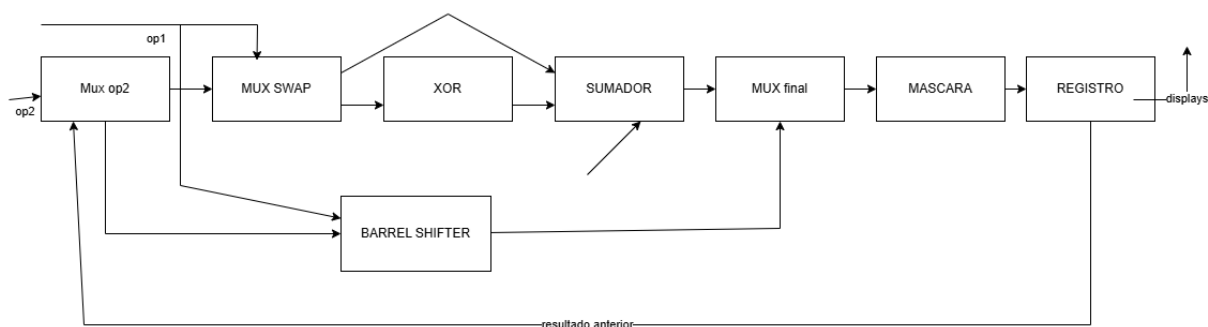
2. DISEÑO GENERAL Y ARQUITECTURA

El circuito implementa una calculadora de 4 bits capaz de operar sobre números con signo representados en complemento a dos. Su comportamiento es el de un acumulador: el resultado de cada operación se almacena en un registro interno y puede reutilizarse como segundo operando de la operación siguiente, lo que permite encadenar cálculos sin volver a ingresar valores intermedios.

Todos los cálculos se realizan con un ancho fijo de cuatro bits. Cuando una operación genera acarreo o desbordamiento, los bits que exceden ese ancho se descartan y se conservan únicamente los cuatro bits menos significativos, que es el comportamiento natural de la aritmética modular en complemento a dos.

Toda la lógica combinacional del circuito se describe exclusivamente mediante compuertas lógicas (and, or, not, xor, nand, nor, xnor, buf), sin recurrir a los operadores aritméticos ni a las construcciones de alto nivel del lenguaje.

Figura 1: Diagrama de bloques de la calculadora



El camino de datos está formado por los siguientes bloques:

- **MUX op2** — elige el segundo operando entre el valor externo y el resultado almacenado.
- **MUX SWAP** — entrega los operandos al sumador en orden natural o intercambiado, lo que distingue la resta de la resta inversa.
- **XOR** — invierte el segundo operando cuando la operación es una resta.
- **Sumador de 4 bits** — construido con cuatro sumadores completos; resuelve la suma y ambas restas.

- **Barrel shifter** — desplaza el primer operando entre 0 y 3 posiciones, en la dirección indicada.
- **MUX final y máscara** — selecciona entre la rama aritmética y la de desplazamiento, y fuerza el valor cero cuando la operación es el reinicio.
- **Registro de 4 bits** — almacena el resultado cuando se activa la confirmación, y lo realimenta al MUX op2.

La decisión de diseño principal fue utilizar un único sumador para las cuatro operaciones aritméticas. La resta se implementa como $A + \neg B + 1$ en complemento a dos, y la resta inversa es la misma operación con los operandos intercambiados a la entrada. El reinicio no requiere sumador, sino una máscara que fuerza la salida a cero. Esto permite que el circuito contenga un solo sumador de 4 bits en lugar de tres.

La calculadora ejecuta seis operaciones, seleccionadas con un código de 3 bits:

Código	Operación	Resultado	Descripción
000	Reinicio	0000	Fuerza a cero el resultado almacenado.
001	Suma	$A+B$	Suma los dos operandos.
010	Resta	$A-B$	Resta B a A usando complemento a dos.
011	Resta inversa	$B-A$	Intercambia los operandos y resta.
100	Shift left	$A \ll B[1:0]$	Desplaza A entre 0 y 3 posiciones a la izquierda, rellenando con ceros.
101	Shift right	$A \gg B[1:0]$	Desplaza A entre 0 y 3 posiciones a la derecha, rellenando con ceros.

El segundo operando puede venir del valor que ingresa el usuario o del resultado guardado de la operación anterior. Esa segunda opción permite encadenar cálculos sin volver a ingresar el valor intermedio.

La interacción con los cuatro botones se organiza en cuatro estados de 2 bits. Cada pulso de confirmación avanza al estado siguiente, y desde el estado que muestra el resultado una nueva confirmación vuelve al inicio.

Estado	Función	Acción principal
00	Selección de operación	SW1 y SW2 recorren los códigos 000 a 101.
01	Ingreso op1	SW1 incrementa y SW2 disminuye el primer operando.

10	Ingreso op2	SW1 y SW2 modifican op2; SW4 permite usar el resultado anterior.
11	Mostrar resultado	Se muestra el resultado; SW3 vuelve al estado 00.

3. TABLAS DE VERDAD, K-MAPS Y EXPRESIONES BOOLEANAS

Esta sección deriva la lógica de cada bloque combinacional. El orden va desde las piezas más básicas hacia las que se construyen con ellas.

3.1 Multiplexor 2:1

Es el bloque más utilizado del diseño. Entrega la entrada a cuando sel = 1 y la entrada b cuando sel = 0.

sel	a	b	y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

Mapa de Karnaugh:

Y	00	01	11	10
sel = 0	0	1	1	0
sel = 1	0	0	1	1

Expresión booleana:

$$Y = \neg \text{sel} \cdot B + \text{sel} \cdot A$$

3.2 Sumador completo

Suma dos bits y el acarreo de entrada, produciendo la suma y el acarreo hacia la etapa siguiente.

A	B	Cin	S	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Mapa de Karnaugh de S (B Cin):

S	00	01	11	10
A = 0	0	1	0	1
A = 1	1	0	1	0

Expresión booleana:

$$S = \neg A \cdot \neg B \cdot \text{Cin} + \neg A \cdot B \cdot \neg \text{Cin} + A \cdot \neg B \cdot \neg \text{Cin} + A \cdot B \cdot \text{Cin}$$

Mapa de Karnaugh de Cout (B Cin):

Cout	00	01	11	10
A = 0	0	0	1	0
A = 1	0	1	1	1

Expresión booleana:

$$\text{Cout} = A \cdot B + A \cdot \text{Cin} + B \cdot \text{Cin}$$

3.3 Sumador y restador de 4 bits

No requiere un mapa de Karnaugh propio: se construye encadenando cuatro sumadores completos, donde el acarreo de salida de cada etapa alimenta el de entrada de la siguiente. La resta se obtiene del mismo circuito. En complemento a dos: $A - B = A + (\text{no})B + 1$.

Cuando sub vale 0, la XOR deja pasar B sin cambios y el acarreo inicial es cero, por lo que se ejecuta la suma. Cuando sub vale 1, la XOR invierte cada bit de b y el acarreo inicial aporta el +1, por lo que se ejecuta una resta.

3.4 Decodificador de control

Genera las cinco señales que gobiernan el camino de datos a partir del código de operación. Los códigos 110 y 111 no están asignados a ninguna operación, por lo que se tratan como condiciones irrelevantes.

op2	op1	op0	Operación	zero	swap	sub	is_shift	dir
0	0	0	Reinicio	1			0	
0	0	1	+	0	0	0	0	
0	1	0	A - B	0	0	1	0	
0	1	1	B - A	0	1	1	0	
1	0	0	shift left	0			1	0
1	0	1	shift right	0			1	1
1	1	0	-----					
1	1	1	-----					

Mapa de Karnaugh zero (op1 op0):

zero	00	01	11	10
op2 = 0	1	0	0	0
op2 = 1	0	0		

Expresión booleana:

$$\text{zero} = \neg \text{op2} \cdot \neg \text{op1} \cdot \neg \text{op0}$$

Mapa de Karnaugh sub (op1 op0):

sub	00	01	11	10
op2 = 0	0	0	1	1
op2 = 1	0	0		

Expresión booleana:
sub = op1

Mapa de Karnaugh swap (op1 op0):

swap	00	01	11	10
op2 = 0	0	0	1	0
op2 = 1	0	0		

Expresión booleana:
swap = op1 · op0

4. IMPLEMENTACIÓN MEDIANTE COMPUERTAS

Cada expresión booleana obtenida en la sección anterior se traduce a instancias de primitivas de compuerta de Verilog. La sintaxis es **compuerta** (salida, entrada1, entrada2) donde el primer argumento corresponde siempre a la salida.

Multiplexor 2:1

Implementa la expresión $Y = \neg sel \cdot b + sel \cdot a$ mediante un inversor, dos compuertas AND y una OR.

```
module mux2_1 (
    input wire a,
    input wire b,
    input wire sel,
    output wire y
);
    wire nsel, t1, t2;

    not (nsel, sel);
    and (t1, sel, a);
    and (t2, nsel, b);
    or (y, t1, t2);
endmodule
```

Sumador completo

Implementa $S = A \oplus B \oplus Cin$ y $Cout = AB + (A \oplus B) \cdot Cin$. La señal intermedia ab corresponde a la XOR compartida entre ambas salidas, lo que reduce el bloque de seis a cinco compuertas.

```
module full_adder (
    input wire a,
    input wire b,
    input wire cin,
    output wire s,
    output wire cout
);
    wire ab, w1, w2;

    xor (ab, a, b);
    xor (s, ab, cin);
    and (w1, a, b);
    and (w2, ab, cin);
    or (cout, w1, w2);
endmodule
```

Suma, resta y resta inversa

Las tres operaciones aritméticas se resuelven en este bloque. Las XOR de entrada aplican $b_i \oplus sub$, invirtiendo el segundo operando cuando la operación es una resta, y la misma señal sub ingresa como acarreo inicial, aportando el +1 del complemento a dos. El acarreo de salida de la última etapa se descarta, lo que produce el truncamiento a cuatro bits. La resta inversa emplea este mismo circuito, con los operandos intercambiados a la entrada por el multiplexor de swap.

```
module add_sub4 (
    input wire [3:0] a,
    input wire [3:0] b,
    input wire      sub,
    output wire [3:0] s,
    output wire      cout
);
    wire [3:0] be;      // b efectivo: invertido si sub = 1
    wire c0, c1, c2;

    xor (be[0], b[0], sub);
    xor (be[1], b[1], sub);
    xor (be[2], b[2], sub);
    xor (be[3], b[3], sub);

    full_adder fa0 (.a(a[0]), .b(be[0]), .cin(sub), .s(s[0]), .cout(c0));
    full_adder fa1 (.a(a[1]), .b(be[1]), .cin(c0), .s(s[1]), .cout(c1));
    full_adder fa2 (.a(a[2]), .b(be[2]), .cin(c1), .s(s[2]), .cout(c2));
    full_adder fa3 (.a(a[3]), .b(be[3]), .cin(c2), .s(s[3]), .cout(cout));
endmodule
```

Desplazamientos

Cada etapa del barrel shifter es una fila de multiplexores que reproduce las expresiones de 2.5. Las entradas constantes 1'b0 corresponden al relleno de ceros que ingresa por el extremo. El desplazamiento a la derecha se implementa de forma simétrica, y un multiplexor final selecciona entre ambas direcciones según `dir`.

```

module shift_left4 (
    input wire [3:0] x,
    input wire [1:0] cant,
    output wire [3:0] z
);
    wire [3:0] y;

    // Etapa 1 — desplaza 1 posición si cant[0] = 1
    mux2_1 e1_3 (.a(x[2]), .b(x[3]), .sel(cant[0]), .y(y[3]));
    mux2_1 e1_2 (.a(x[1]), .b(x[2]), .sel(cant[0]), .y(y[2]));
    mux2_1 e1_1 (.a(x[0]), .b(x[1]), .sel(cant[0]), .y(y[1]));
    mux2_1 e1_0 (.a(1'b0), .b(x[0]), .sel(cant[0]), .y(y[0]));

    // Etapa 2 — desplaza 2 posiciones si cant[1] = 1
    mux2_1 e2_3 (.a(y[1]), .b(y[3]), .sel(cant[1]), .y(z[3]));
    mux2_1 e2_2 (.a(y[0]), .b(y[2]), .sel(cant[1]), .y(z[2]));
    mux2_1 e2_1 (.a(1'b0), .b(y[1]), .sel(cant[1]), .y(z[1]));
    mux2_1 e2_0 (.a(1'b0), .b(y[0]), .sel(cant[1]), .y(z[0]));
endmodule

```

Reinicio e integración

El módulo `alu4` conecta todos los bloques anteriores según el diagrama de la Figura 1. Los multiplexores `msx` y `msy` realizan el intercambio de operandos, y las cuatro compuertas AND finales aplican la máscara $r_i = pre_i \cdot \text{-zero}$, que implementa la operación de reinicio sin necesidad de una entrada adicional en el multiplexor de selección.

```

module alu4 (
    input wire [3:0] a,      // op1
    input wire [3:0] b,      // op2 efectivo
    input wire [2:0] op,
    output wire [3:0] r
);

    wire zero, swap, sub, is_shift, dir, nzero, cdesc;
    wire [3:0] x, y, add, sh, pre;

    ctrl_decode ctrl (.op(op), .zero(zero), .swap(swap), .sub(sub),
                     .is_shift(is_shift), .dir(dir));

    mux2_4 msx (.a(b), .b(a), .sel(swap), .y(x));
    mux2_4 msy (.a(a), .b(b), .sel(swap), .y(y));

    add_sub4 asu (.a(x), .b(y), .sub(sub), .s(add), .cout(cdesc));
    barrel_shift4 bs (.x(a), .cant(b[1:0]), .dir(dir), .z(sh));

    mux2_4 mf (.a(sh), .b(add), .sel(is_shift), .y(pre));

    not (nzero, zero);
    and (r[0], pre[0], nzero);

```

```

    and (r[1], pre[1], nzero);

    and (r[2], pre[2], nzero);

    and (r[3], pre[3], nzero);

endmodule

```

5. SIMULACIÓN Y RESULTADOS

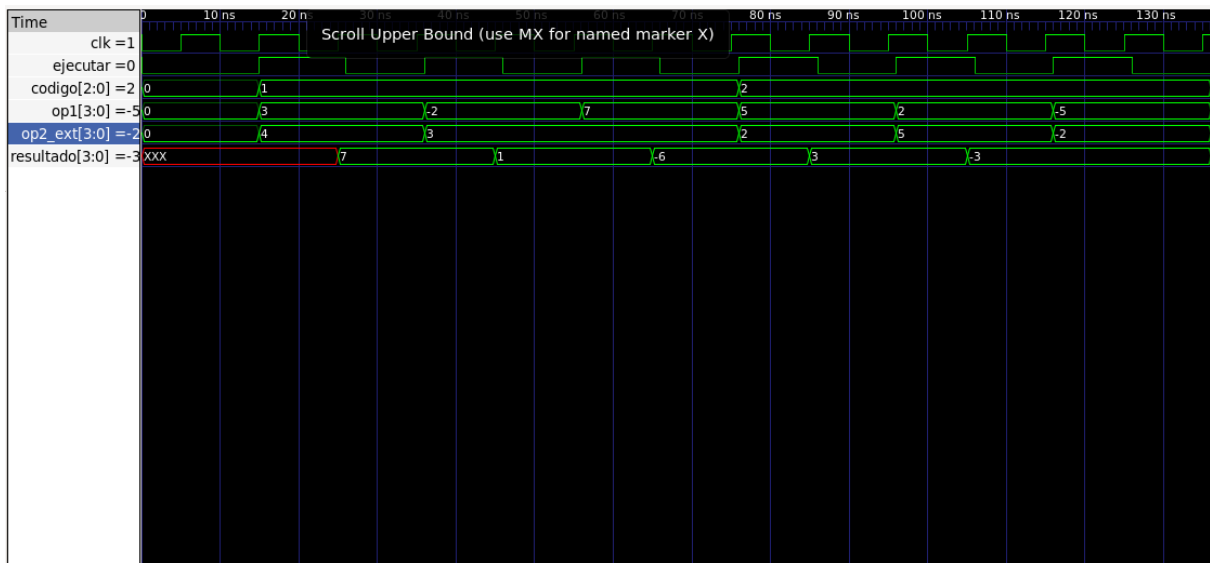
5.1 Estrategia de verificación

Cada módulo se probó por separado antes de integrarlo. Para los bloques pequeños se recorrieron todas las combinaciones posibles de entrada: en la ALU, con dos operandos de 4 bits y seis operaciones, son 1536 casos. Como revisar esa cantidad de formas de onda a mano es imposible, el testbench compara automáticamente el resultado del circuito contra un modelo de referencia escrito con los operadores nativos de Verilog. Esos operadores están prohibidos en el diseño, pero no en el testbench, que no se sintetiza.

5.2 Resultado de simulación

Testbench	Que verifica	Errores
alu4_tb	1536 casos	0
calculator_tb	6 operaciones con el flujo de botones	0
debouncer_tb	Filtro de botones	0
calculator_top:tb	Sistema completo	0

También se ejecutó el testbench publicado en el foro del curso, que instancia el diseño con los nombres de puerto del enunciado. Los seis casos pasaron sin modificar el diseño. Uno de ellos es $7 + 3$, que da 1010: en complemento a dos eso vale -6 y no 10, porque con cuatro bits con signo el rango va de -8 a $+7$ y el resultado se sale. El testbench del curso espera ese valor, lo que confirma que el comportamiento implementado es el correcto.



Captura de GTKWave con op, op1, op2 y resultado en decimal con signo.

5.3 Síntesis

El diseño se sintetizó con Yosys usando synth_ice40, sin errores y sin advertencias de latches accidentales. El diseño ocupa cerca de una quinta parte de las celdas disponibles, así que cabe con holgura. La mayor parte del costo viene del barrel shifter, que calcula los dos sentidos en paralelo.

5.4 Pruebas en placa

El bitstream se generó con Yosys, nextpnr, icepack e iceprog, y se cargó en la Go Board. Los casos verificados sobre el hardware fueron:

Operación	op1	op2	Esperado	Observado
Suma	5	2	7	7
Suma (desborde)	5	3	-8	-8
Resta	5	3	2	2
Resta	3	5	-2	-2
Resta inversa	3	5	2	2

Reinicio			0	0
-----------------	--	--	---	---

También se comprobó el botón de resultado anterior, usando el valor guardado como segundo operando de una nueva operación. El caso de $5 + 3$ se muestra como -8 porque el patrón 1000, en complemento a dos con cuatro bits, vale -8 . El circuito calcula bien: lo que ocurre es un desbordamiento con signo, que es el comportamiento que define el enunciado al pedir que se conserven solo los cuatro bits menos significativos.

El diseño cumple con lo especificado: las seis operaciones funcionan correctamente tanto en simulación como sobre la FPGA, y toda la lógica combinacional está construida únicamente con compuertas. Las dos decisiones que más redujeron el circuito fueron compartir un solo sumador entre las cuatro operaciones aritméticas y aprovechar las condiciones irrelevantes de los códigos 110 y 111 al simplificar el decodificador de control, que quedó en apenas dos compuertas lógicas.